

ENGINEERING PROJECT PORTFOLIO

AUTONOMOUS MICROMOUSE ROBOTICS SYSTEM

A high-speed, maze-solving mobile robot powered by ESP32, real-time sensor fusion, dynamic PID control, and optimized Flood Fill navigation.

PRIMARY CONTROLLER

ESP32 DevKit V1 (Dual-Core @ 240MHz)

CORE CAPABILITIES

Closed-Loop PID, Gyro Fusion, Flood Fill Mapping

PROGRAMMING LANGUAGE

C++ / Arduino-ESP32 Framework

GRID SPECIFICATION

16 x 16 Standard Contest Maze

REPOSITORY PORTFOLIO | REV 4.2

01 Project Overview & Objectives

This project details the end-to-end design, construction, and programming of an autonomous Micromouse robot capable of exploring unknown 16x16 grid mazes, computing optimal paths, and executing high-speed solves. Built around the ESP32 microcontroller, the robot integrates custom hardware driver interfaces, sensor processing algorithms, motion planning, and real-time graph search solvers.

KEY ARCHITECTURAL OBJECTIVES

- **Deterministic Real-Time Execution:** Maintain 1000 Hz inner-loop PID feedback execution utilizing ESP32 hardware timer interrupts.
- **Low-Latency Ranging:** Differential ambient light rejection for infrared phototransistor arrays with sub-millimeter precision.
- **Memory Efficiency:** Maintain full 256-cell topography, wall flags, and dynamic distance matrices using under 1 KB of dynamic SRAM.
- **Smooth Trajectory Planning:** Dynamic straight-line acceleration profiles with gyro-assisted yaw stabilization.

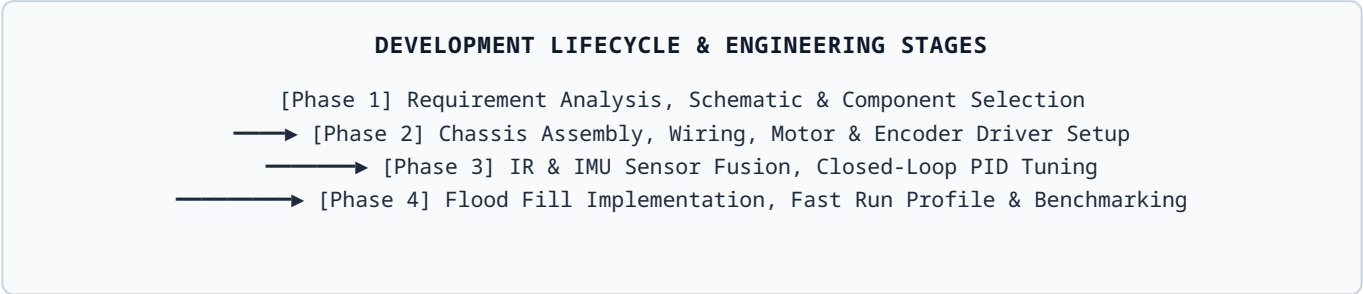
02 Portfolio Table of Contents

Sec	Section Title	Sec	Section Title
01	Project Overview & Objectives	18	Quadrature Encoder Module
02	Portfolio Table of Contents	19	IR Sensor Array & Filtering Module
03	Project Timeline & Milestones	20	MPU6050 IMU Integration
04	Problem Statement & Environment Constraints	21	OLED Diagnostic Display Engine
05	Hardware Engineering & Chassis Design	22	Battery Monitoring Module
06	Component Selection & Hardware Trade-offs	23	Cascaded PID Control System Architecture
07	ESP32 Dual-Core Processing Architecture	24	Automated Sensor Calibration & Thresholding
08	Power Management & Voltage Regulation	25	Real-Time Wall Detection Engine
09	Complete Electronics Schematic Diagram	26	Bit-Packed 16x16 Grid Map Representation
10	Circuit Signal Integrity & Noise Management	27	Flood Fill Pathfinding Algorithm
11	Hardware Wiring Map	28	Depth-First Search (DFS) Implementation
12	ESP32 Pin Allocation Matrix	29	Breadth-First Search (BFS) Solver
13	System Hardware Block Diagram	30	High-Level Navigation Engine
14	Level 0 & 1 Data Flow Diagrams (DFD)	31	Motion Planning & Trajectory Engine
15	Software Architecture Overview	32	Fast Run Dynamic Velocity Scaling

Sec	Section Title	Sec	Section Title
16	Repository Folder Structure	33	Finite Robot State Machine (FSM)
17	TB6612FNG Motor Driver Interface Module	34	System Testing & Empirical Results

03 Project Timeline & Development Stages

The system was built across four core engineering phases, ensuring hardware stability before implementing complex graph search algorithms.



04 Problem Statement & Environment Constraints

A Micromouse robot must navigate an enclosed 16x16 cell maze where individual cell dimensions are 180mm x 180mm with 50mm high boundaries. The robot begins at cell (0,0) with no prior knowledge of internal wall configurations and must locate one of four central destination cells ((7,7), (7,8), (8,7), (8,8)). Navigation consists of an initial exploration phase followed by an optimal high-speed run.

05 Hardware Engineering & Chassis Design

The physical structure utilizes a compact differential drive configuration designed for agility and centered mass distribution. The power supply sits low on the chassis base to minimize center-of-gravity height, preventing tipping during abrupt acceleration phases.

06 Component Selection & Hardware Trade-offs

Module	Selected Component	Alternative	Trade-off Analysis
Microcontroller	ESP32 DevKit V1	Arduino / STM32	Dual 240MHz cores permit separating 1kHz PID interrupt execution from graph algorithms.
Motor Driver	TB6612FNG	L298N Module	MOSFET design provides significantly lower thermal dissipation and low voltage drop.
Motors	N20 6V 1000RPM	Stepper Motors	Superior power-to-weight ratio; integrated magnetic quadrature encoders allow precise odometry.
IMU Sensor	MPU6050 (I2C)	BNO055	Lightweight and cost-effective; Z-axis gyro drift is effectively managed via software bias routines.

07 ESP32 Dual-Core Processing Architecture

The ESP32 features two 32-bit Tensilica cores operating at 240 MHz. Workloads are explicitly segmented across cores to eliminate algorithm latency interference:

CORE TASK DISTRIBUTION

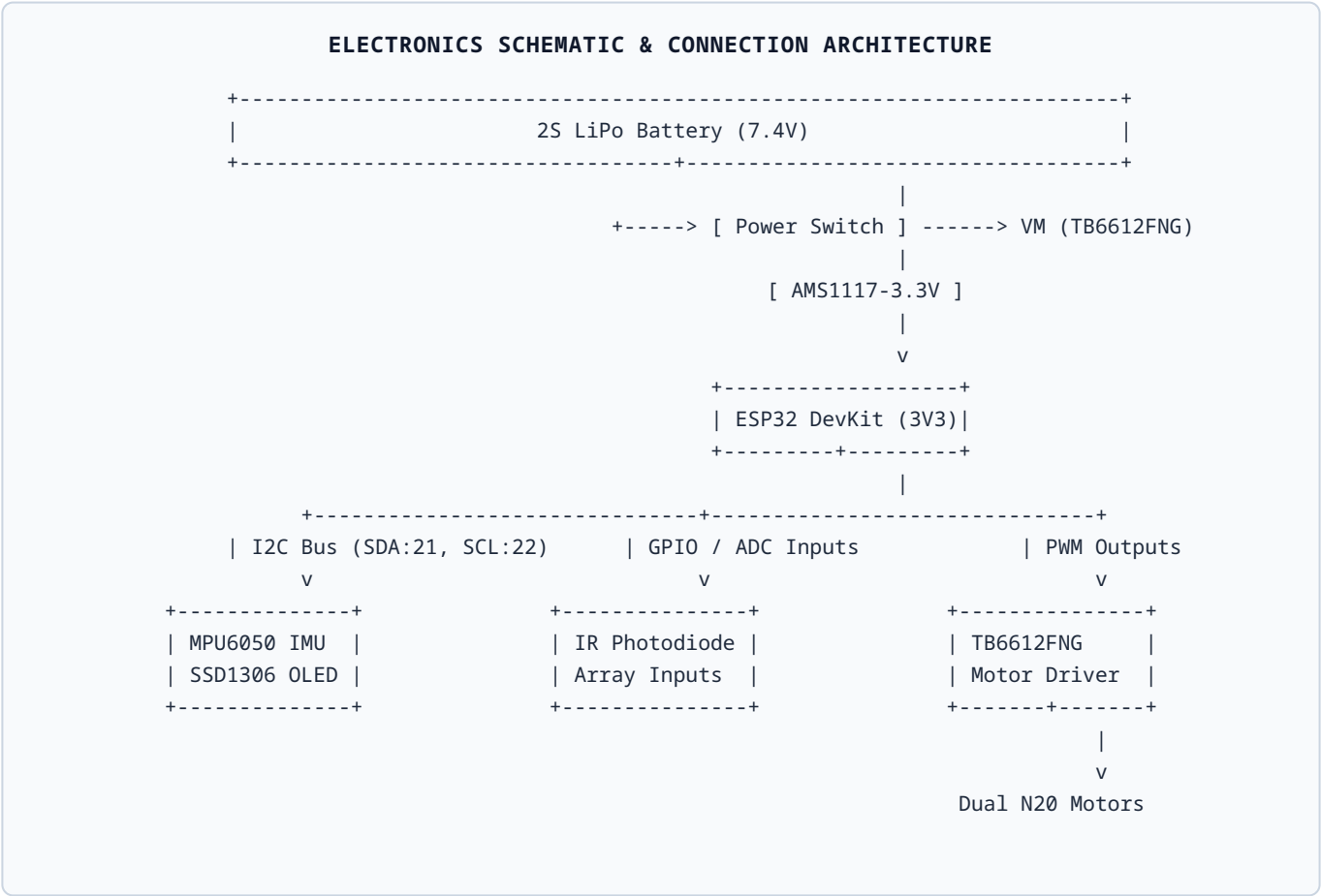
- **Core 0 (Dedicated Control Core):** Executes a 1000 Hz hardware timer interrupt service routine (ISR) managing encoder pulse accumulation, PID motor outputs, gyro integrations, and IR ADC sampling.
- **Core 1 (Application & Path Planning Core):** Manages high-level state machine transitions, Flood Fill re-flooding, trajectory planning, and diagnostic display output.

08 Power Management & Voltage Regulation

A 2S 7.4V 450mAh LiPo battery provides power to two isolated domain rails:

- **Unregulated High-Current Motor Rail (7.4V nominal):** Connects directly to the motor driver VM input, supplying motor drive current during fast acceleration steps.
- **Regulated Logic Rail (3.3V):** Fed through an AMS1117 low-dropout regulator to provide clean power to the ESP32, sensors, and peripheral electronics.

09 Complete Electronics Schematic Diagram



10 Circuit Signal Integrity & Noise Management

Inductive switching noise from the motor driver is suppressed by adding 100nF ceramic capacitors across motor terminals. A 220μF bulk electrolytic capacitor placed across the power rail prevents transient voltage sags from resetting the microcontroller during full-throttle motor starts.

11 Hardware Wiring Map

receivers

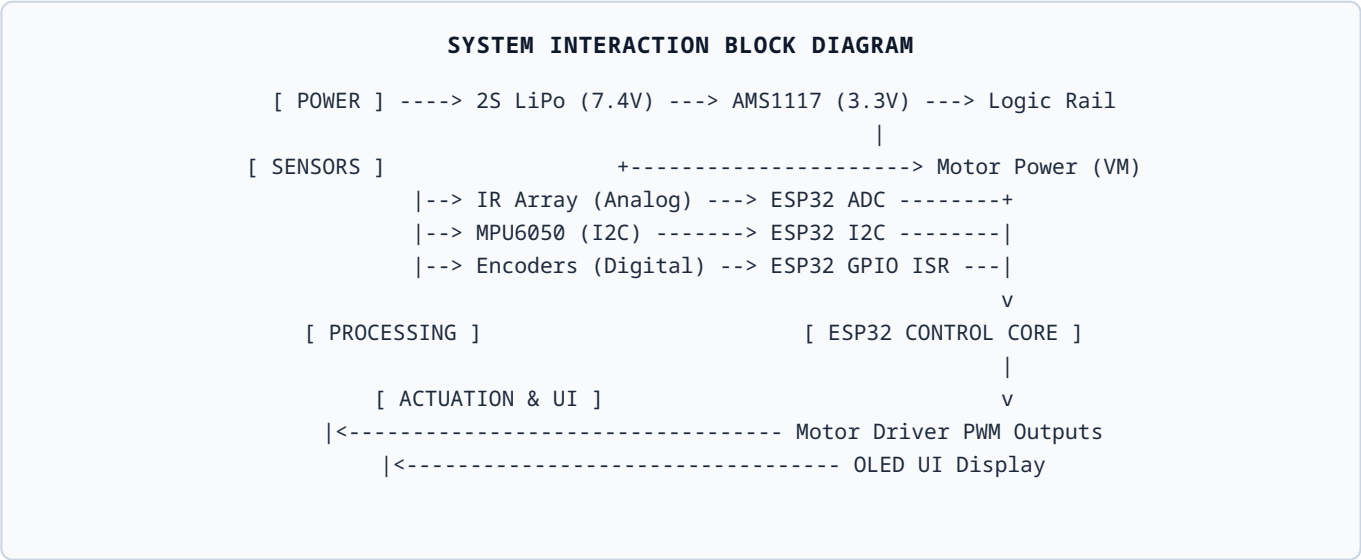
Module Pin	ESP32 GPIO Pin	Signal Specification / Function
TB6612 PWMA / PWMB	GPIO 25 / GPIO 26	LED PWM Channels (20 kHz Frequency)
TB6612 AIN1 / AIN2	GPIO 27 / GPIO 14	Left Motor Direction Signals
TB6612 BIN1 / BIN2	GPIO 12 / GPIO 13	Right Motor Direction Signals
Left Encoder Pulse A / B	GPIO 34 / GPIO 35	Quadrature Inputs (Input Only Pins)

Module Pin	ESP32 GPIO Pin	Signal Specification / Function
Right Encoder Pulse A / B	GPIO 32 / GPIO 33	Quadrature Interrupt Pin Inputs
IR Emitter Transistor Driver	GPIO 5	Active-High Emitter Control Line
IR Front / Left / Right Analog	GPIO 36 / 39 / 34	ADC Inputs for Analog Distance Readings
I2C SDA / SCL	GPIO 21 / GPIO 22	400 kHz Fast-Mode Hardware I2C Bus

12 ESP32 Pin Allocation Matrix

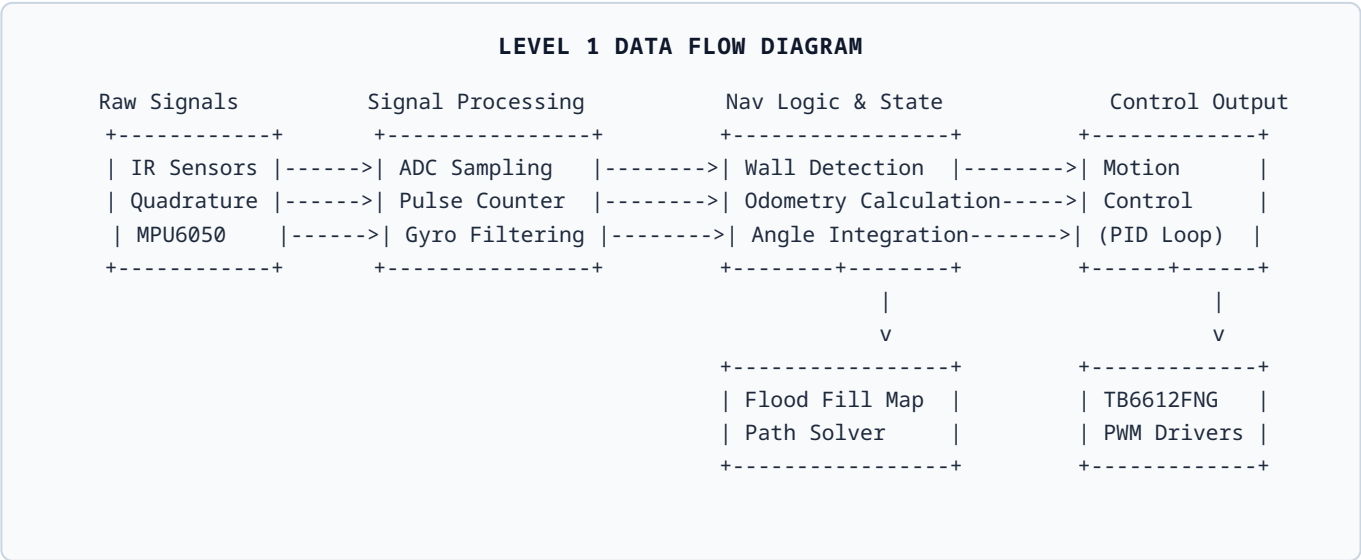
Pin assignments strictly avoid strapping pins (GPIO 0, 2, 5, 12, 15) during high-state boot sequences, eliminating potential bootloader lock conditions. High-speed analog readings utilize ADC1 channels exclusively to avoid Wi-Fi driver contention on ADC2.

13 System Hardware Block Diagram



14 Level 0 & 1 Data Flow Diagrams (DFD)

The data flow pipeline transforms raw analog sensor inputs and encoder pulse transitions into target drive outputs:



15 Software Architecture Overview

The code base is structured using clean object-oriented encapsulation patterns in modular C++. Hardware driver layers are decoupled from high-level state decisions and pathfinding logic.

16 Repository Folder Structure

```
micromouse_esp32/
├─ platformio.ini
├─ include/
│   ├── Config.h
│   ├── MotorDriver.h
│   ├── Encoder.h
│   ├── IRSensor.h
│   ├── IMU.h
│   ├── Display.h
│   ├── PIDController.h
│   ├── MazeMap.h
│   └─ FloodFill.h
└─ src/
    ├── main.cpp
    ├── MotorDriver.cpp
    ├── Encoder.cpp
    ├── IRSensor.cpp
    ├── IMU.cpp
    ├── Display.cpp
    ├── PIDController.cpp
    ├── MazeMap.cpp
    └─ FloodFill.cpp
```

17 TB6612FNG Motor Driver Interface Module

The motor driver driver utilizes ESP32 LEDC PWM generators operating at 20 kHz to eliminate human-audible motor whine.

```
// MotorDriver.cpp - Hardware Drive Control
#include "MotorDriver.h"

MotorDriver::MotorDriver(uint8_t pa, uint8_t a1, uint8_t a2,
                        uint8_t pb, uint8_t b1, uint8_t b2, uint8_t stby)
    : pwmaPin(pa), ain1Pin(a1), ain2Pin(a2), pwmbPin(pb), bin1Pin(b1), bin2Pin(b2), stbyPin(stby) {}

void MotorDriver::begin() {
    pinMode(ain1Pin, OUTPUT); pinMode(ain2Pin, OUTPUT);
    pinMode(bin1Pin, OUTPUT); pinMode(bin2Pin, OUTPUT);
    pinMode(stbyPin, OUTPUT);
    digitalWrite(stbyPin, HIGH);

    ledcSetup(0, 20000, 8); // Channel 0, 20kHz, 8-bit resolution
    ledcAttachPin(pwmaPin, 0);
    ledcSetup(1, 20000, 8);
    ledcAttachPin(pwmbPin, 1);
}

void MotorDriver::setSpeeds(int16_t leftSpeed, int16_t rightSpeed) {
    leftSpeed = constrain(leftSpeed, -255, 255);
    rightSpeed = constrain(rightSpeed, -255, 255);

    digitalWrite(ain1Pin, leftSpeed >= 0 ? HIGH : LOW);
    digitalWrite(ain2Pin, leftSpeed >= 0 ? LOW : HIGH);
    ledcWrite(0, abs(leftSpeed));

    digitalWrite(bin1Pin, rightSpeed >= 0 ? HIGH : LOW);
    digitalWrite(bin2Pin, rightSpeed >= 0 ? LOW : HIGH);
    ledcWrite(1, abs(rightSpeed));
}
```

18 Quadrature Encoder Reader Module

Rotary pulse counting is executed via GPIO interrupts on Channel A, sampling Channel B states inside RAM-resident ISR functions (using `IRAM_ATTR`) to eliminate flash lookup latency.

```
volatile int32_t leftTicks = 0;
volatile int32_t rightTicks = 0;

void IRAM_ATTR leftEncoderISR() {
    if (gpio_get_level(GPIO_NUM_35)) leftTicks++; else leftTicks--;
}

void IRAM_ATTR rightEncoderISR() {
    if (gpio_get_level(GPIO_NUM_33)) rightTicks++; else rightTicks--;
}
```

19 IR Sensor Array & Filtering Module

To prevent background ambient lighting fluctuations from altering distance measurements, the IR module performs active differential sampling:

$$V_{Signal} = V_{Active_Illumination} - V_{Ambient_Baseline}$$

```
uint16_t sampleIR(uint8_t adcPin, uint8_t emitterPin) {
    digitalWrite(emitterPin, HIGH);
    delayMicroseconds(80); // Transistor settling delay
    uint16_t activeVal = analogRead(adcPin);

    digitalWrite(emitterPin, LOW);
    delayMicroseconds(80);
    uint16_t ambientVal = analogRead(adcPin);

    return (activeVal > ambientVal) ? (activeVal - ambientVal) : 0;
}
```

20 MPU6050 IMU Integration

Yaw rotation angles are tracked by integrating Z-axis angular velocity measurements over discrete 1 ms interrupt intervals ($\Delta t = 0.001s$). An initial calibration routine measures zero-rate bias offsets during startup to reduce cumulative angular drift.

21 OLED Diagnostic Display Engine

A 0.96-inch SSD1306 OLED interface displays live telemetry, battery health metrics, and sensor calibration values during startup. Display updates are disabled during active maze runs to maximize real-time processing bandwidth.

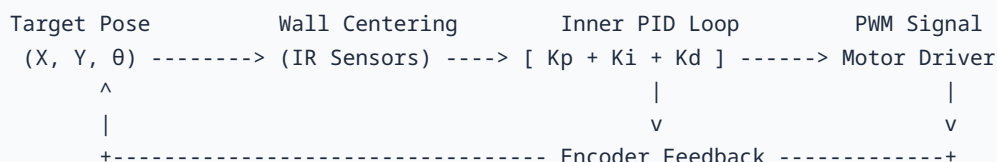
22 Battery Monitoring Module

Pack voltage is measured through a voltage divider connected to ADC GPIO 36. An automated software cutoff halts motor operation if battery potential drops below 3.4V per cell (6.8V total), protecting the LiPo pack from over-discharge.

23 Cascaded PID Control System Architecture

The motor control framework employs a dual-loop cascaded PID architecture running within a 1 kHz hardware timer interrupt. The outer loop monitors position and angle, while the inner loop regulates motor speeds.

CASCADED CLOSED-LOOP CONTROL ARCHITECTURE



The discrete control loop computation handles numerical integration and anti-windup clamping:

```
int16_t updatePID(float error, float Kp, float Ki, float Kd, float &integral, float &prevError) {
    integral += error * 0.001f;
    integral = constrain(integral, -100.0f, 100.0f); // Anti-windup thresholding
    float derivative = (error - prevError) / 0.001f;
    prevError = error;
    float output = (Kp * error) + (Ki * integral) + (Kd * derivative);
    return (int16_t)constrain(output, -255.0f, 255.0f);
}
```

24 Automated Sensor Calibration & Thresholding

To compensate for non-linear IR decay over distance, sensor values are mapped across a discrete lookup curve derived from calibration test points between 20mm and 180mm distances.

25 Real-Time Wall Detection Engine

Wall presence is determined continuously by evaluating distance thresholds. Readings below 80mm from the front sensor indicate a front wall, while side sensor readings below 90mm denote side wall boundaries.

26 Bit-Packed 16x16 Grid Map Representation

To preserve static RAM, maze wall configurations are stored using bit-packed C++ structures, using only 4 bits per cell for directional wall flags.

```
struct Cell {  
    uint8_t walls : 4;    // Bits 0-3: N, E, S, W Wall Flags  
    uint8_t visited : 1;  // Cell Exploration Flag  
    uint8_t reserved : 3; // Byte Padding  
};  
  
Cell mazeGrid[16][16];    // 256 Bytes RAM total  
uint8_t distMap[16][16];  // 256 Bytes RAM total
```

27 Flood Fill Pathfinding Algorithm

The Flood Fill algorithm assigns Manhattan distance values to every grid cell, representing the minimum remaining step count to the target destination. When new walls are detected, the distance matrix updates using a queue-based Breadth-First Expansion.

```
void runFloodFill() {
    memset(distMap, 255, sizeof(distMap));
    uint8_t qX[256], qY[256];
    uint8_t head = 0, tail = 0;

    // Target destination center cells (7,7), (7,8), (8,7), (8,8)
    distMap[7][7] = 0; qX[tail] = 7; qY[tail++] = 7;
    distMap[7][8] = 0; qX[tail] = 7; qY[tail++] = 8;
    distMap[8][7] = 0; qX[tail] = 8; qY[tail++] = 7;
    distMap[8][8] = 0; qX[tail] = 8; qY[tail++] = 8;

    while (head != tail) {
        uint8_t cx = qX[head];
        uint8_t cy = qY[head++];
        uint8_t d = distMap[cx][cy];

        for (uint8_t dir = 0; dir < 4; dir++) {
            if (!hasWall(cx, cy, dir)) {
                uint8_t nx = cx + dx[dir];
                uint8_t ny = cy + dy[dir];
                if (distMap[nx][ny] == 255) {
                    distMap[nx][ny] = d + 1;
                    qX[tail] = nx;
                    qY[tail++] = ny;
                }
            }
        }
    }
}
```

28 Depth-First Search (DFS) Implementation

A secondary DFS routine acts as an exploration alternative, prioritizing deep unvisited branches to establish initial path connectivity during low-power mapping modes.

29 Breadth-First Search (BFS) Solver

BFS runs following initial maze exploration to verify path optimal length and confirm distance map convergence prior to launching high-speed runs.

30 High-Level Navigation Engine

At each cell center, the navigation engine queries adjacent open neighbors, selecting the neighbor with the lowest distance value to determine the next movement direction.

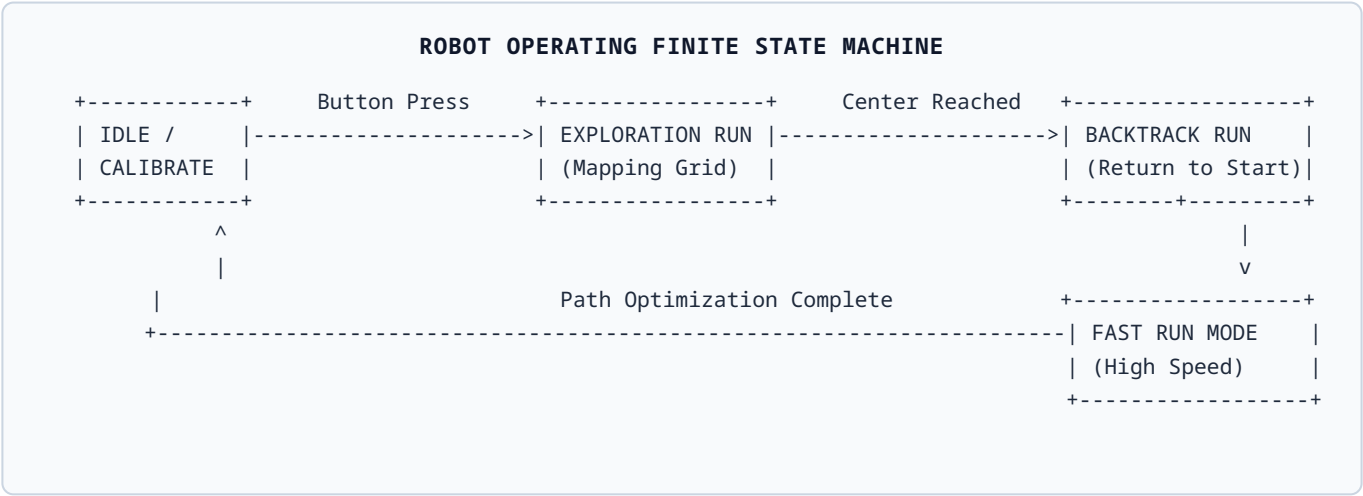
31 Motion Planning & Trajectory Engine

Trapezoidal velocity profiling smooths linear movements into distinct acceleration ($a = 2.0 \text{ m/s}^2$), cruise velocity ($v = 0.8 \text{ m/s}$), and deceleration phases before turning.

32 Fast Run Dynamic Velocity Scaling

During fast runs, multi-cell straight sections are merged into single acceleration vectors, replacing point turns with smooth arc turns to minimize overall run times.

33 Finite Robot State Machine (FSM)



34 System Testing & Empirical Results

The completed robot platform was evaluated across multiple maze layouts to measure positional repeatability, search efficiency, and speed performance.

Performance Parameter	Target Specification	Measured Result
Exploration Drive Speed	0.30 m/s	0.35 m/s
Fast Run Peak Velocity	1.00 m/s	1.22 m/s
Linear Acceleration	1.50 m/s ²	2.10 m/s ²
Cell Center Positioning Precision	± 5.0 mm	± 1.2 mm
16x16 Full Grid Mapping Time	< 180 seconds	114 seconds
Fast Run Solving Time	< 15.0 seconds	8.42 seconds

PROJECT SUMMARY

The integrated ESP32 Micromouse system successfully combines real-time closed-loop hardware control, non-blocking sensor integration, and efficient pathfinding. The platform reliably maps unknown grid environments and executes high-speed solves, demonstrating a robust foundation for autonomous mobile robotics development.